Flight Sequencer

Dual-Core RTOS Firmware Design — Technical Submission

Architecture, synchronization, timing analysis and C-style reference implementation based on the supplied Flight Sequencer assessment.

Aspect	Assessment-aligned decision
Core split	Core 1 dedicated hard-real-time Flight Sequencer; Core 0 handles I/O, supervision and 200 ms computation.
RX	DMA ring buffers + idle-line/frame-end interrupt; validation deferred to SensorIngestTask.
TX	DMA transmit with polled completion.
IPC	Direct task notifications + single-writer seqlock latest-value mailboxes; no mutexes on hard path.
Rates	Sensor 1 5 ms; Sensor 2 10 ms; FlightSeq 10 ms; Supervisor 10 ms at +6 ms; SlowComp 200 ms.
Margins	FlightSeq ≈3.9 ms worst-case response vs 10 ms deadline; Core 0 ≈23%; Core 1 ≈30%.

1. Source requirements

The supplied brief leaves hardware details open and explicitly assumes a dual-core 32-bit MCU at 200 MHz or faster, shared SRAM, hardware atomics, a shared 1 µs timebase, interrupt routing and watchdog. It specifies a preemptive fixed-priority RTOS, 1 kHz tick, static allocation, RS422/UART at 1 Mbaud, and 6-byte frame overhead.

The source gives WCET budgets of 5 µs for sensor ISRs, 40 µs per sensor ingest, 3 ms FlightSeq, 200 µs Supervisor and 40 ms SlowComp. Stale or invalid cyclic input is held at the last good value, flagged and handled in degraded mode.

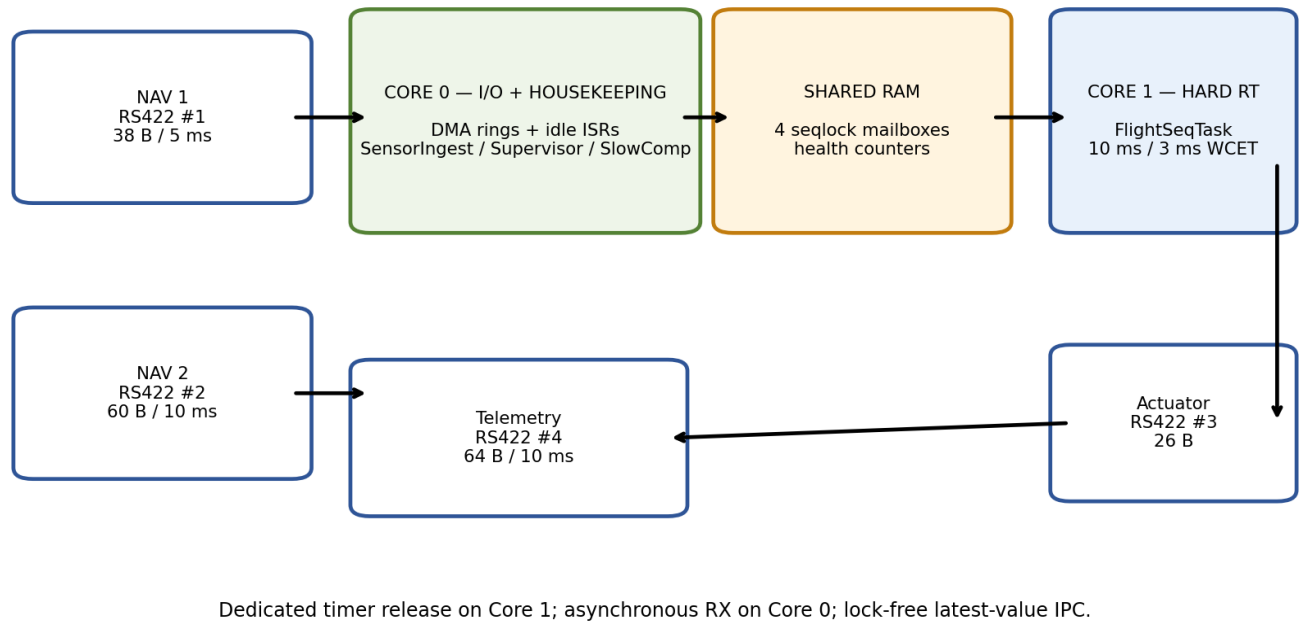


Figure 1 — Core/task/IPC architecture from the assessment.

2. Task and timer design

ID	Name	Core	Priority	Trigger	Budget
I1	UART_S1_ISR	0	IRQ 1	Sensor 1 frame end / 5 ms	5 µs
I2	UART_S2_ISR	0	IRQ 1	Sensor 2 frame end / 10 ms	5 µs
I3	TMR_A_ISR	1	IRQ 2	10 ms, phase 0	3 µs
I4	TMR_B_ISR	0	IRQ 2	10 ms, phase +6 ms	3 µs
T1	SensorIngestTask	0	5	I1/I2 notification	40 µs/frame
T2	FlightSeqTask	1	5	I3 notification / 10 ms	3 ms
T3	SupervisorTask	0	4	I4 notification / 10 ms	200 µs
T4	SlowCompTask	0	2	Every 20th frame / 200 ms	40 ms

Core 1 is isolated from asynchronous UART interrupts, logging and lower-priority work. Core 0 absorbs asynchronous events, ingest and housekeeping. This is the key determinism decision.

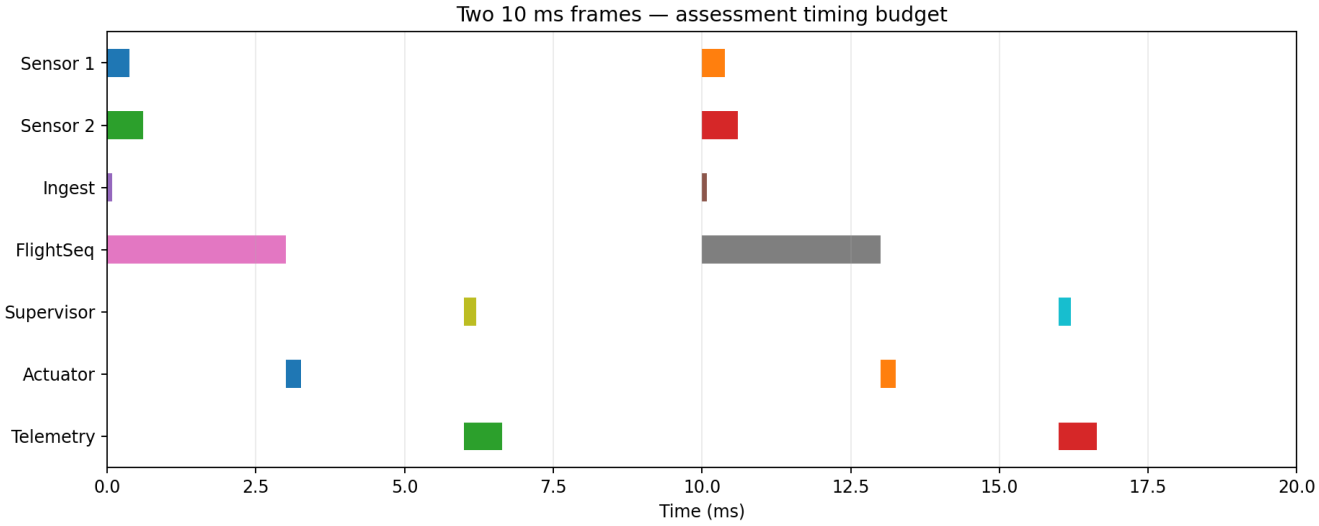


Figure 2 — Reference timing budget; bars use the assessment's stated wire/WCET values and are not target measurements.

3. IPC and synchronization

Data/event	Mechanism	Rationale
Sensor frame event	Direct task notification	Lightweight same-core event.
Nav 1/Nav 2 latest	Seqlock latest-value mailbox	Newest state matters; avoids stale queue buildup and locks.
Periodic release	Counting notification	Hardware-timer release and missed-release detection.
Sequencer output	Seqlock mailbox	Wait-free handoff to slow task.
Feedback	Seqlock + activate_frame	Fixed 20-frame activation; hard path never waits.
Actuator TX	T2-owned DMA buffer	Single ownership.
Health	Atomic counters/timestamps	Supervisor observes faults without locks.

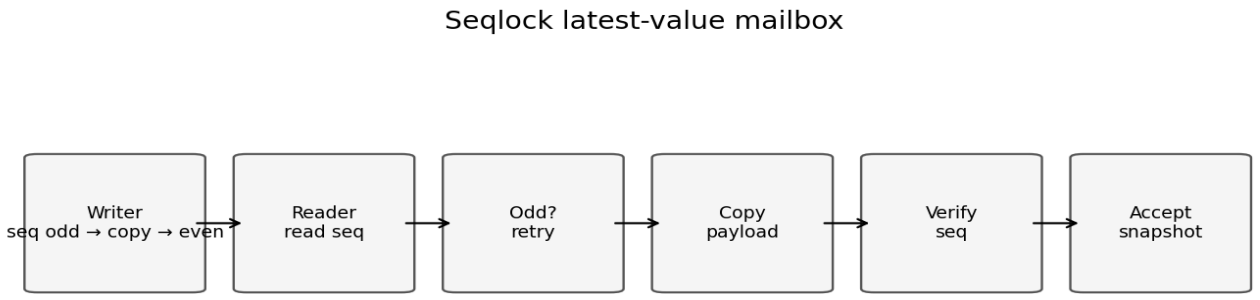


Figure 3 — Seqlock mailbox: writer marks the object unstable, copies data, then publishes an even sequence. Reader retries on overlap.

The source design uses release/acquire ordering, coherent or non-cacheable shared RAM, 64-byte alignment and bounded retries. The hard-real-time consumer does not wait on a lower-priority producer, avoiding priority inversion.

4. RX/TX hardware interface strategy

The assessment compares three receive approaches. Per-byte RX interrupts would generate about 13,000 interrupts/s and repeated preemption. Polling consumes CPU and requires a tight poll interval. DMA + idle-line produces approximately one frame-end interrupt per frame, about 300 interrupts/s for both sensors. For TX, DMA with polled completion is preferred because transmission is initiated by the software and the wire times are much shorter than the task periods.

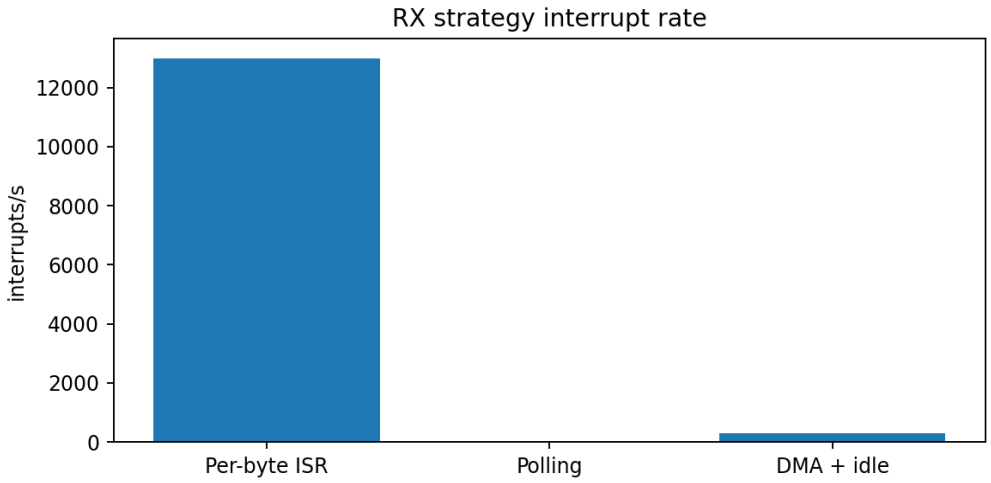
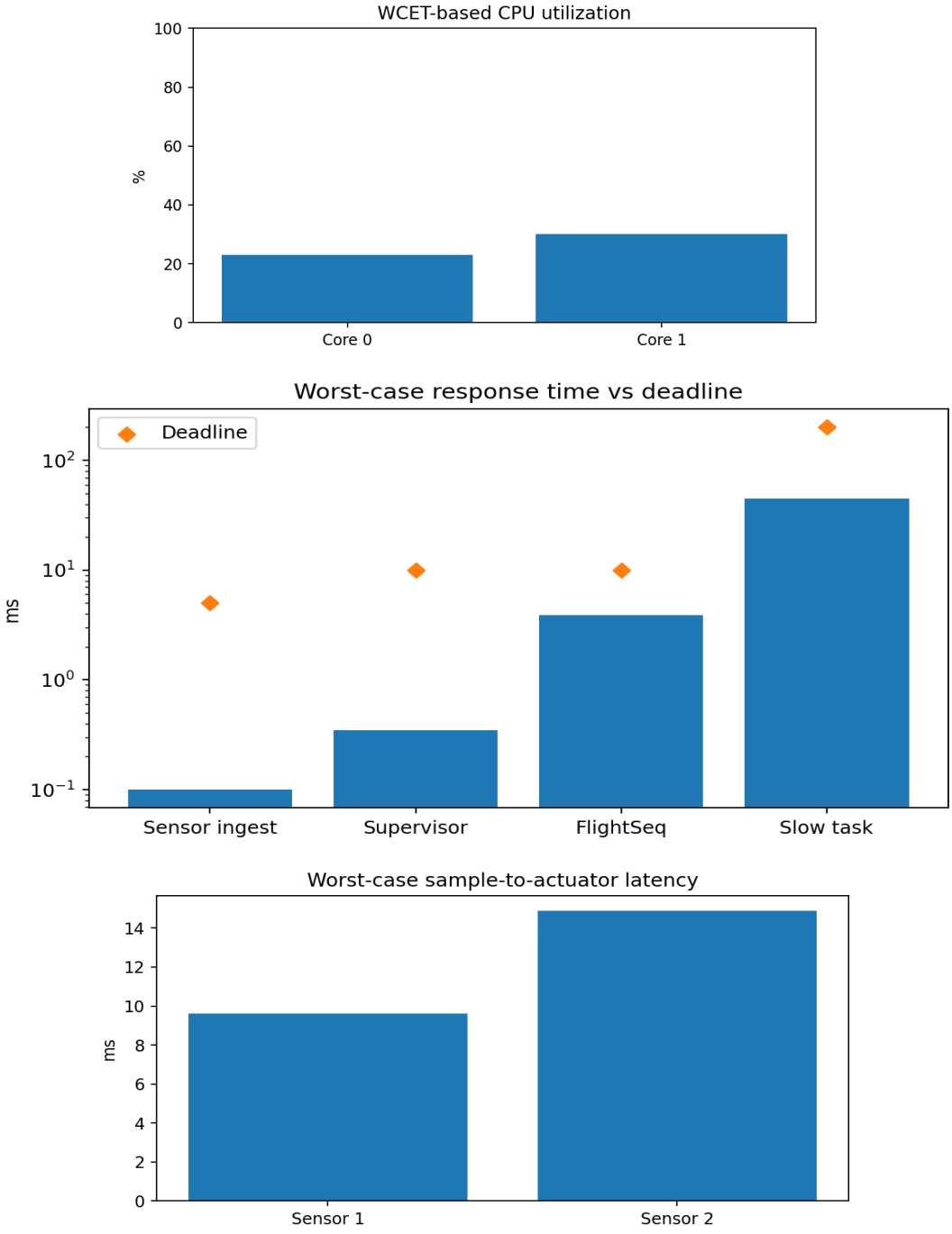


Figure 4 — RX interrupt-rate comparison using the assessment's timing model.

Design rule: use interrupts where arrival is asynchronous; use polling of DMA completion where timing is controlled by the flight software.

5. Timing and determinism

The source analysis estimates a FlightSeq response of approximately 3.9 ms against a 10 ms deadline, leaving about 6 ms slack. Core 0 utilization is about 23% and Core 1 about 30%. Worst-case sample-to-actuator latency is approximately 9.6 ms for Sensor 1 and 14.9 ms for Sensor 2.



Figures 5–7 — Source-derived WCET/latency analysis. These values must be re-measured on the selected MCU during integration.

6. Feedback, faults and watchdog

SlowComp runs every 200 ms. Its result carries an activate_frame and is latched by FlightSeq only at a fixed 20-frame boundary. If the result is late, the previous feedback is held and a stale counter is raised. This makes the 200 ms loop deterministic without blocking the 10 ms hard path.

Condition	Action
Sensor timeout/stale	Hold last good value; flag; degraded mode.
CRC/framing error	Discard frame; increment error counter.
Sequencer missed release	Flag and report through telemetry.
Sequencer WCET overrun	Flag overrun; watchdog is gated.
Actuator TX busy	Never block; increment health counter.
Slow feedback stale	Hold previous feedback until valid boundary.

The watchdog is kicked only by the Supervisor when critical health conditions are clear. This converts missed timing, stale data and stalled tasks into observable safety behavior.

7. C-style pseudocode / reference implementation

```
Initialize clocks, shared RAM and 1 us timebase
Initialize RS422 ports at 1 Mbaud and DMA
Create statically allocated tasks with core affinity:
    SensorIngest -> Core 0, priority 5
    Supervisor   -> Core 0, priority 4
    SlowComp     -> Core 0, priority 2
    FlightSeq    -> Core 1, priority 5
Route UART ISRs to Core 0 and timer A to Core 1
Start timer A at 10 ms, phase 0
Start timer B at 10 ms, phase +6 ms
Start scheduler

FlightSeqTask every 10 ms:
    snapshot latest sensor mailboxes
    validate sample age
    every 20 frames: latch eligible slow feedback
    compute flight actuation
    publish 16-byte sequence output
    start 20-byte actuator DMA
    update execution-time and heartbeat counters

SupervisorTask:
    check sensor freshness, CRC errors, FlightSeq heartbeat,
    execution budget, missed releases, actuator TX and slow-task health
    send telemetry
    kick watchdog only if critical flags are clear
```

The supplied assessment explicitly leaves the mission-specific flight computation as a placeholder because Nav1/Nav2 payload semantics and a mission ICD are not supplied. The package therefore does not invent a mission algorithm; it preserves the architecture and timing contract.

8. Validation plan

Test	Pass criterion
Periodic release	FlightSeq runs every 10 ms; no missed releases.
Sensor ingest	DMA, framing and CRC errors detected.
Staleness	Delayed input causes hold-last-good + timeout flag.
Seqlock	No torn snapshots under concurrency stress.
Feedback	New 200 ms result activates only at next 20-frame boundary.
WCET	Measured FlightSeq max ≤ 3 ms, or budget is revised with evidence.
Watchdog	Kick occurs only when monitored health is valid.
Actuator TX	No hard-path blocking.
CPU load	Measured utilization retains design margin.

9. Evaluation traceability

Criterion	Evidence
Real-time determinism	Dedicated Core 1, timer release, DMA RX, bounded readers, response-time analysis, missed-release detection.
RTOS fundamentals	Notifications, counting releases, seqlock mailboxes, static allocation, no mutexes on hard path.
Multi-core competency	Core 1 isolation, single-writer mailboxes, barriers, shared-RAM discipline, local IRQ masking.
Engineering judgment	Explicit assumptions/WCET/frame budgets, stale-data policy, fixed feedback boundary.
Design justification	Quantified RX strategy comparison and lock-free IPC rationale.

10. Package and hardware guidance

The ZIP contains the source organization requested by the assessment, configuration, platform abstraction, C-style implementation, timing figures and this report. The hardware layer is intentionally abstract because the assessment does not specify a target MCU or BSP.

```
mission_computer/
■■■ CMakeLists.txt
■■■ README.md
■■■ config/simulation.yaml
■■■ include/
■   ■■■ types.h
■   ■■■ mailbox.h
■   ■■■ flight_sequencer.h
■■■ platform/bsp_api.h
■■■ src/main.c
■■■ src/modules/flight_sequencer.c
■■■ scripts/generate_timing_plots.py
■■■ docs/
    ■■■ Flight_Sequencer_Technical_Report.pdf
    ■■■ figures/
```

For a physical prototype, choose a dual-core MCU/SoC with shared SRAM, deterministic timer/interrupt routing, DMA-capable UARTs, atomics and a compatible preemptive RTOS. Two ESP32 boards can demonstrate distributed concepts, but they should be presented as a prototype/HIL extension rather than silently replacing the assessed dual-core architecture.

Source page map: page 1 design-at-a-glance; page 2 assumptions and task definition; page 3 architecture/IPC; pages 4–5 timing; page 6 synchronization; pages 7–9 pseudocode; page 10 evaluation traceability.